CHRIST (Deemed to be University)
Department of Computer Science
Master of Computer Applications

Course: MCA411-2 - PYTHON Programming
LAB Exercise IV –
Implement exceptional handling. Apply a regular expression for string manipulation.

Date of assignment: 30-09-2026
Date of submission: 30-09-2026

Name : Aditya Kumar

Reg. No : 2647203
Class : 2 MCA – B

Domain : Clean Energy Systems & Grid Decarbonization

Lab 4

Lab Exercise IV:

Implement exceptional handling. Apply a regular expression for string manipulation.

A. Develop a Python program to design a domain-specific data extraction and validation pipeline using Regular Expressions and Exception Handling.

Code:

```
'''
# Domain: Clean Energy Systems & Grid Decarbonization
'''

import re


'''
*** 1. CUSTOM EXCEPTIONS ***
'''

# 1. Custom Exception for Battery Overheat
class BatteryOverheatError(Exception):
    pass

# 2. Custom Exception for Wrong Status
class InvalidOperationalStatusError(Exception):
    pass


'''
*** 2. REGULAR EXPRESSION PATTERNS (10 Patterns) ***
'''

# 1. Site ID (like SITE_1, SITE_BLR01)
PAT_SITE_ID = r"^SITE_[A-Z0-9]+$"

# 2. Date (YYYY-MM-DD)
PAT_DATE = r"^\d{4}-(0[1-9]|1[0-2])-(0[1-9]|[12]\d|3[01])$"

# 3. Time (HH:MM or HH:MM:SS)
PAT_TIME = r"^([01]\d|2[0-3]):[0-5]\d(:[0-5]\d)?$"

# 4. Email address
PAT_EMAIL = r"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$"

# 5. Mobile / Phone number (10 digits)
PAT_PHONE = r"^(\+91[\-\s]?)?[6-9]\d{9}$"

# 6. PIN Code (6 digits)
PAT_PINCODE = r"^[1-9]\d{5}$"

# 7. Equipment / Inverter Code (e.g. INV-1001-AB)
```

```
PAT_EQUIP_CODE = r"^INV-[0-9]{4}-[A-Z]{2}$"

# 8. Web URL (http or https)
PAT_GATEWAY_URL = r"^https?://[a-zA-Z0-9.-]+(:\d+)?(/.*)?$"

# 9. Numeric value (Panel count)
PAT_INTEGER = r"^\d+$"

# 10. Decimal measurement (Power, Temp, SoC)
PAT_DECIMAL = r"^\d+(\.\d+)?$"


'''
*** 3. STORAGE & VALIDATION PIPELINE ***
'''

# Default sample records already stored
valid_telemetry_records = [
    {
        "site_id": "SITE_1",
        "date": "2026-09-25",
        "time": "10:30",
        "email": "north_admin@grid.com",
        "phone": "9876543210",
        "pin": "560001",
        "inv_code": "INV-1001-AA",
        "gateway_url": "https://solar-north.grid.in",
        "panel_count": 50,
        "power_kw": 150.0,
        "temp_c": 32.0,
        "soc_pct": 75.0,
        "status": "ONLINE",
        "avg_kw_per_panel": 3.0
    },
    {
        "site_id": "SITE_2",
        "date": "2026-09-25",
        "time": "11:15",
        "email": "south_tech@grid.com",
        "phone": "9123456780",
        "pin": "560002",
        "inv_code": "INV-1002-BB",
        "gateway_url": "https://solar-south.grid.in",
        "panel_count": 30,
        "power_kw": 90.0,
        "temp_c": 40.0,
        "soc_pct": 25.0,
        "status": "CHARGING",
        "avg_kw_per_panel": 3.0
    }
]


# Function to validate record before adding
def validate_telemetry_log(raw_log):
    # Exception 3: TypeError (check if dict)
    if not isinstance(raw_log, dict):
        raise TypeError("[Site: Unknown | Field: raw_log] Record must be a
dictionary.")
```

```
    # Safely identify Site ID for reporting
    sid = str(raw_log.get("site_id", "UNKNOWN")).strip().upper()

    # Exception 4: KeyError (check if any field is missing)
    fields = [
        "site_id", "date", "time", "email", "phone",
        "pin", "inv_code", "gateway_url", "panel_count",
        "power_kw", "temp_c", "soc_pct", "status"
    ]
    for f in fields:
        if f not in raw_log:
            raise KeyError(f"[Site: {sid} | Field: '{f}'] Missing mandatory
field.")

    # Exception 5: ValueError (check empty values)
    for k, v in raw_log.items():
        if isinstance(v, str) and v.strip() == "":
            raise ValueError(f"[Site: {sid} | Field: '{k}'] Field cannot be
empty.")

    # 1. Regex check for Site ID
    if not re.match(PAT_SITE_ID, raw_log["site_id"].strip()):
        raise ValueError(f"[Site: {sid} | Field: 'site_id'] Invalid Site ID
format! Expected like SITE_1 or SITE_BLR01.")

    # 2. Regex check for Date
    if not re.match(PAT_DATE, raw_log["date"].strip()):
        raise ValueError(f"[Site: {sid} | Field: 'date'] Invalid Date
'{raw_log['date']}'! Please use YYYY-MM-DD.")

    # 3. Regex check for Time
    if not re.match(PAT_TIME, raw_log["time"].strip()):
        raise ValueError(f"[Site: {sid} | Field: 'time'] Invalid Time
'{raw_log['time']}'! Please use HH:MM.")

    # 4. Regex check for Email
    if not re.match(PAT_EMAIL, raw_log["email"].strip()):
        raise ValueError(f"[Site: {sid} | Field: 'email'] Invalid Email address
'{raw_log['email']}'.")

    # 5. Regex check for Phone
    if not re.match(PAT_PHONE, raw_log["phone"].strip()):
        raise ValueError(f"[Site: {sid} | Field: 'phone'] Invalid Phone
'{raw_log['phone']}'. Must be 10 digits.")

    # 6. Regex check for PIN Code
    if not re.match(PAT_PINCODE, raw_log["pin"].strip()):
        raise ValueError(f"[Site: {sid} | Field: 'pin'] Invalid PIN
'{raw_log['pin']}'. Must be 6 digits.")

    # 7. Regex check for Inverter Code
    if not re.match(PAT_EQUIP_CODE, raw_log["inv_code"].strip()):
        raise ValueError(f"[Site: {sid} | Field: 'inv_code'] Invalid Inverter Code
'{raw_log['inv_code']}'. Expected format: INV-XXXX-YY.")

    # 8. Regex check for URL
    if not re.match(PAT_GATEWAY_URL, raw_log["gateway_url"].strip()):
```

```
        raise ValueError(f"[Site: {sid} | Field: 'gateway_url'] Invalid URL
'{raw_log['gateway_url']}'. Must start with http:// or https://.")

    # 9. Regex check for Panel count (Integer)
    if not re.match(PAT_INTEGER, str(raw_log["panel_count"]).strip()):
        raise ValueError(f"[Site: {sid} | Field: 'panel_count'] Panel count
'{raw_log['panel_count']}' must be a whole number.")

    # 10. Regex check for Decimals (Power, Temp, SoC)
    for key in ["power_kw", "temp_c", "soc_pct"]:
        if not re.match(PAT_DECIMAL, str(raw_log[key]).strip()):
            raise ValueError(f"[Site: {sid} | Field: '{key}'] Invalid decimal
reading '{raw_log[key]}'.")

    # Convert to numbers
    panels = int(raw_log["panel_count"])
    power = float(raw_log["power_kw"])
    temp = float(raw_log["temp_c"])
    soc = float(raw_log["soc_pct"])

    # Exception 6: ZeroDivisionError
    if panels == 0:
        raise ZeroDivisionError(f"[Site: {sid} | Field: 'panel_count'] Cannot
calculate power per panel when panel count is 0 (division by zero).")

    avg_power = power / panels

    # Custom Exception 1: Overheat check
    if temp > 45.0:
        raise BatteryOverheatError(f"[Site: {sid} | Field: 'temp_c'] High
temperature alert: {temp}°C is greater than 45°C limit!")

    # Custom Exception 2: Status check
    valid_states = ["ONLINE", "CHARGING", "DISCHARGING", "STANDBY"]
    if raw_log["status"].strip().upper() not in valid_states:
        raise InvalidOperationalStatusError(f"[Site: {sid} | Field: 'status']
Invalid status '{raw_log['status']}'. Must be ONLINE, CHARGING, DISCHARGING, or
STANDBY.")

    return {
        "site_id": sid,
        "date": raw_log["date"].strip(),
        "time": raw_log["time"].strip(),
        "email": raw_log["email"].strip(),
        "phone": raw_log["phone"].strip(),
        "pin": raw_log["pin"].strip(),
        "inv_code": raw_log["inv_code"].strip(),
        "gateway_url": raw_log["gateway_url"].strip(),
        "panel_count": panels,
        "power_kw": power,
        "temp_c": temp,
        "soc_pct": soc,
        "status": raw_log["status"].strip().upper(),
        "avg_kw_per_panel": round(avg_power, 2)
    }


# Process and catch exceptions
def process_record(raw_log):
```

```
    try:
        clean = validate_telemetry_log(raw_log)
        valid_telemetry_records.append(clean)
        print(f"Success: Record {clean['site_id']} added successfully.")
        return True
    except KeyError as e:
        print("KeyError Caught:", e)
    except TypeError as e:
        print("TypeError Caught:", e)
    except ValueError as e:
        print("ValueError Caught:", e)
    except ZeroDivisionError as e:
        print("ZeroDivisionError Caught:", e)
    except BatteryOverheatError as e:
        print("BatteryOverheatError Caught:", e)
    except InvalidOperationalStatusError as e:
        print("InvalidOperationalStatusError Caught:", e)
    # Exception 7: General Exception
    except Exception as e:
        print("General Exception Caught:", e)
    return False


'''
*** 4. RUNTIME EXCEPTIONS DEMONSTRATION ***
'''

def demonstrate_runtime_exceptions():
    print("\n--- Additional Exception Demos ---")

    # Exception 8: IndexError
    try:
        readings = [10.5, 20.0]
        print(readings[5])
    except IndexError as e:
        print("[Field: readings[5]] IndexError Caught: List index out of range.")

    # Exception 9: AttributeError
    try:
        num = 100
        num.append(20)
    except AttributeError as e:
        print("[Field: num.append] AttributeError Caught: 'int' object has no
attribute 'append'.")

    # Exception 10: NameError
    try:
        total = unknown_variable + 10
    except NameError as e:
        print("[Field: unknown_variable] NameError Caught: Variable is not
defined.")


'''
*** 5. Console Menu ***
'''

def view_records():
    print("\n--- All Valid Records ---")
```

```
    if not valid_telemetry_records:
        print("No records found.")
        return

    for r in valid_telemetry_records:
        print(f"Site ID: {r['site_id']} | Date: {r['date']} {r['time']} | Status:
{r['status']}")
        print(f"  Inverter: {r['inv_code']} | URL: {r['gateway_url']}")
        print(f"  Contact: {r['email']} | Phone: {r['phone']} | PIN: {r['pin']}")
        print(f"  Panels: {r['panel_count']} | Power: {r['power_kw']} kW (Avg:
{r['avg_kw_per_panel']} kW/panel)")
        print(f"  Battery: {r['soc_pct']}% | Temp: {r['temp_c']}°C")
        print("-" * 45)


def add_record():
    print("\n--- Add New Telemetry Record ---")
    log = {
        "site_id": input("Enter Site ID (e.g. SITE_3): "),
        "date": input("Enter Date (YYYY-MM-DD): "),
        "time": input("Enter Time (HH:MM): "),
        "email": input("Enter Email: "),
        "phone": input("Enter Phone: "),
        "pin": input("Enter PIN Code: "),
        "inv_code": input("Enter Inverter Code (e.g. INV-2001-CD): "),
        "gateway_url": input("Enter URL (e.g. https://site.in): "),
        "panel_count": input("Enter Panel Count: "),
        "power_kw": input("Enter Power in kW: "),
        "temp_c": input("Enter Temp in °C: "),
        "soc_pct": input("Enter Battery SoC %: "),
        "status": input("Enter Status (ONLINE/CHARGING/DISCHARGING/STANDBY): ")
    }
    process_record(log)


def run_tests():
    print("\n--- Running Validation Tests ---")

    # Test 1: Wrong Email Regex
    print("\n1. Testing Wrong Email:")
    t1 = {
        "site_id": "SITE_TEST1", "date": "2026-09-30", "time": "12:00",
        "email": "not_an_email", "phone": "9876543210", "pin": "560001",
        "inv_code": "INV-1001-AA", "gateway_url": "https://test.com",
        "panel_count": "20", "power_kw": "50.0", "temp_c": "30.0",
        "soc_pct": "60.0", "status": "ONLINE"
    }
    process_record(t1)

    # Test 2: ZeroDivisionError
    print("\n2. Testing 0 Panels (ZeroDivisionError):")
    t2 = t1.copy()
    t2["site_id"] = "SITE_TEST2"
    t2["email"] = "admin@grid.in"
    t2["panel_count"] = "0"
    process_record(t2)

    # Test 3: Custom BatteryOverheatError
    print("\n3. Testing Battery Temperature > 45°C:")
```

```
    t3 = t1.copy()
    t3["site_id"] = "SITE_TEST3"
    t3["email"] = "admin@grid.in"
    t3["temp_c"] = "50.0"
    process_record(t3)

    # Test 4: Custom InvalidOperationalStatusError
    print("\n4. Testing Invalid Status:")
    t4 = t1.copy()
    t4["site_id"] = "SITE_TEST4"
    t4["email"] = "admin@grid.in"
    t4["status"] = "FAULTY"
    process_record(t4)

    # Test 5: Missing Key (KeyError)
    print("\n5. Testing Missing Field:")
    t5 = t1.copy()
    t5["site_id"] = "SITE_TEST5"
    t5["email"] = "admin@grid.in"
    del t5["pin"]
    process_record(t5)


# Main Menu Loop
while True:
    print("\n--- SOLAR TELEMETRY PIPELINE ---")
    print("1. View Valid Records")
    print("2. Add Record Interactively")
    print("3. Run Validation Tests")
    print("4. Demonstrate Runtime Exceptions")
    print("5. Exit")

    choice = input("Enter choice (1-5): ").strip()
    if choice == "1":
        view_records()
    elif choice == "2":
        add_record()
    elif choice == "3":
        run_tests()
    elif choice == "4":
        demonstrate_runtime_exceptions()
    elif choice == "5":
        print("Exiting...")
        break
    else:
        print("Invalid choice.")
```

Output:

```
--- SOLAR TELEMETRY PIPELINE ---
1. View Valid Records
2. Add Record Interactively
3. Run Validation Tests
4. Demonstrate Runtime Exceptions
5. Exit
Enter choice (1-5):
```

```
Enter choice (1-5): 1

--- All Valid Records ---
Site ID: SITE_1 | Date: 2026-09-25 10:30 | Status: ONLINE
  Inverter: INV-1001-AA | URL: https://solar-north.grid.in
  Contact: north_admin@grid.com | Phone: 9876543210 | PIN: 560001
  Panels: 50 | Power: 150.0 kW (Avg: 3.0 kW/panel)
  Battery: 75.0% | Temp: 32.0°C
-----------------------------------------
Site ID: SITE_2 | Date: 2026-09-25 11:15 | Status: CHARGING
  Inverter: INV-1002-BB | URL: https://solar-south.grid.in
  Contact: south_tech@grid.com | Phone: 9123456780 | PIN: 560002
  Panels: 30 | Power: 90.0 kW (Avg: 3.0 kW/panel)
  Battery: 25.0% | Temp: 40.0°C
-----------------------------------------
```

```
--- Add New Telemetry Record ---
Enter Site ID (e.g. SITE_3): SITE_3
Enter Date (YYYY-MM-DD): 2026-10-10
Enter Time (HH:MM): 01
Enter Email: aa@aaa.com
Enter Phone: 1234567890
Enter PIN Code: 521063
Enter Inverter Code (e.g. INV-2001-CD): INV-2001-CD
Enter URL (e.g. https://site.in): https://site.in
Enter Panel Count: 4
Enter Power in kW: 150
Enter Temp in °C: 45
Enter Battery SoC %: 80
Enter Status (ONLINE/CHARGING/DISCHARGING/STANDBY): STANDBY
ValueError Caught: [Site: SITE_3 | Field: 'time'] Invalid Time '01'! Please use HH:MM.
```

```
Enter choice (1-5): 3

--- Running Validation Tests ---

1. Testing Wrong Email:
ValueError Caught: [Site: SITE_TEST1 | Field: 'email'] Invalid Email address 'not_an_email'.

2. Testing 0 Panels (ZeroDivisionError):
ZeroDivisionError Caught: [Site: SITE_TEST2 | Field: 'panel_count'] Cannot calculate power per panel when panel count is 0 (division by zero).

3. Testing Battery Temperature > 45°C:
BatteryOverheatError Caught: [Site: SITE_TEST3 | Field: 'temp_c'] High temperature alert: 50.0°C is greater than 45°C limit!

4. Testing Invalid Status:
InvalidOperationalStatusError Caught: [Site: SITE_TEST4 | Field: 'status'] Invalid status 'FAULTY'. Must be ONLINE, CHARGING, DISCHARGING, or STANDBY.

5. Testing Missing Field:
KeyError Caught: "[Site: SITE_TEST5 | Field: 'pin'] Missing mandatory field."
```

```
Enter choice (1-5): 4

--- Additional Exception Demos ---
[Field: readings[5]] IndexError Caught: List index out of range.
[Field: num.append] AttributeError Caught: 'int' object has no attribute 'append'.
[Field: unknown_variable] NameError Caught: Variable is not defined.
```